



Ministerul Educației și Cercetării
Universitatea "OVIDIUS" Constanța
Facultatea de Matematică și Informatică
Specializarea Informatică

Gym-Connect – Aplicație web pentru gestionarea antrenamentelor fizice

Lucrare de licență

Coordonator științific:
Lect.univ.dr. Dorin Iordache

Absolvent:
Dode Andrei - Robert

Constanța
2026

Abstract

Tipul lucrării: sinteză și cercetare aplicativă

Lucrarea de față descrie procesul de proiectare și construire a aplicației web **Gym-Connect**, un instrument dedicat sportivilor care practică antrenamente de forță și doresc să își urmărească progresul în mod structurat. Ideea a pornit dintr-o frustrare reală: majoritatea aplicațiilor similare de pe piață sunt fie prea complicate, fie cer bani pentru funcții de bază. Gym-Connect își propune să fie altfel – gratuit, simplu și accesibil direct din browser, fără instalare.

Din punct de vedere tehnic, proiectul urmărește patru obiective principale: construirea unei arhitecturi client-server funcționale în care frontenidul React comunică cu un server Express prin cereri HTTP autentificate; implementarea autentificării utilizatorilor prin Clerk; realizarea unui modul de înregistrare a antrenamentelor în timp real, cu suport pentru calculul greutatea și indicatori de intensitate precum RIR și 1RM estimat; și, nu în ultimul rând, construirea unui modul de analiză a progresului care folosește regresia liniară pentru a estima evoluția pe termen scurt și a detecta automat platourile de performanță.

Lucrarea este organizată în șase capitole. Primele două stabilesc contextul – de ce există această problemă și ce soluții mai există pe piață. Următoarele două intră în detaliile tehnice ale soluției propuse. Ultimele două prezintă aplicația concret, așa cum arată și funcționează ea, și trag concluziile.

Cuvinte cheie: aplicație web, antrenamente de forță, progressive overload, React, TypeScript, Express.js, Prisma, PostgreSQL, Clerk, analiză progres, 1RM, RIR.

Cuprins

Abstract	i
Abstract	i
Cuprins	ii
1 Introducere și Motivație	2
1.1 Contextul problemei	2
1.2 Motivație personală	2
1.3 Obiectivele lucrării	3
1.4 Contribuții originale	3
1.5 Structura lucrării	4
2 Starea actuală a domeniului	5
2.1 Aplicații existente de monitorizare a antrenamentelor	5
2.1.1 MyFitnessPal	5
2.1.2 Strong App	6
2.1.3 Hevy	6
2.1.4 Jefit	7
2.2 Analiză comparativă	7

2.3	Poziționarea Gym-Connect	8
3	Tehnologii și elemente teoretice utilizate	10
3.1	React	10
3.2	TypeScript	11
3.3	Vite	11
3.4	Express.js	11
3.5	Prisma ORM	12
3.6	PostgreSQL și Neon DB	12
3.7	Clerk	12
3.8	Tailwind CSS	13
3.9	Recharts	13
3.10	Zod	13
3.11	React Router	14
3.12	React Hook Form	14
4	Soluția propusă și arhitectura sistemului	15
4.1	Arhitectura generală	15
4.2	Modelul de date	16
4.2.1	Modelul SplitTemplate	16
4.2.2	Modelul WorkoutSession	16
4.2.3	Modelul WorkoutSet	17
4.2.4	Relațiile dintre modele	18
4.3	API-ul REST	18
4.3.1	Endpoint-uri pentru rutine (/api/splits)	18
4.3.2	Endpoint-uri pentru sesiuni (/api/sessions)	18
4.4	Autentificarea și securitatea	19
4.4.1	Fluxul de autentificare	19

4.4.2	Middleware-ul de autentificare	19
4.4.3	Protecția rutelor în frontend	20
4.5	Diagrama Use-Case	20
5	Prezentarea aplicației	21
5.1	Pagina de start (Landing Page)	23
5.2	Autentificare și înregistrare	23
5.3	Gestionarea rutinelor (Split)	23
5.3.1	Crearea unei rutine	23
5.3.2	Selectorul de exerciții	23
5.4	Sesiunea de antrenament (Workout)	23
5.4.1	Cronometrul și indicatorul de progres	23
5.4.2	Gestionarea seturilor	23
5.4.3	Calculatorul de plăci	23
5.4.4	Indicatori de intensitate: RIR și 1RM	23
5.5	Istoricul antrenamentelor (History)	23
5.6	Modulul de analitică (Analytics)	23
5.6.1	Graficul volumului săptămânal	23
5.6.2	Predicția 1RM și regresia liniară	23
5.6.3	Detecția platourilor	23
6	Concluzii	24
6.1	Rezultate obținute	24
6.2	Limitările soluției actuale	24
6.3	Direcții de dezvoltare ulterioară	24
	Referințe bibliografice	25

Capitolul 1

Introducere și Motivație

1.1 Contextul problemei

Trăim într-o perioadă în care aproape orice aspect al vieții cotidiene a fost digitalizat – de la cumpărături la servicii bancare. Cu toate acestea, în sala de forță lucrurile stau surprinzător de diferit. Mulți sportivi amatori încă își notează antrenamentele într-un caiet fizic sau, mai rău, se bazează pe memorie. Consecința este mereu aceeași: după două luni de antrenament, nu mai știi exact ce greutate ai folosit săptămâna trecută și nu poți să îți dai seama dacă ai progresat sau bătut pasul pe loc.

Primele aplicații dedicate fitness-ului au apărut în jurul anului 2009, simultan cu explozia smartphone-urilor [1]. De atunci, piața a crescut considerabil, însă direcția de dezvoltare a majorității produselor a urmat același tipar: funcții de bază gratuite, restul în spatele unui abonament lunar. Rezultatul este că un sportiv care vrea să vadă un grafic simplu al progresului său trebuie să plătească, deseori fără să știe exact ce primește în schimb.

Principiul **progressive overload** – ideea că mușchii cresc când îi supui treptat la efort tot mai mare – este recunoscut în literatură ca fundamentul oricărui program de antrenament eficient [2]. Aplicarea lui corectă presupune însă să știi întotdeauna de unde ai pornit și unde ai ajuns. Fără un istoric clar al antrenamentelor, progressive overload devine aproape imposibil de aplicat în practică.

1.2 Motivație personală

Ideea acestui proiect a apărut dintr-o situație cu care m-am confruntat personal. Ca student la Informatică care frecventează sala de forță de câțiva ani, am încercat mai multe aplicații de tracking – de la MyFitnessPal la Strong App. Problema era întotdeauna aceeași: fie aplicația îmi cerea bani pentru funcții pe care le consideră elementare, fie era atât de încărcată de opțiuni încât mă descuraja s-o mai deschid după antrenament, când ești obosit și nu ai chef de meniuri complicate.

La un moment dat mi-am zis că, dacă tot învăț să construiesc aplicații web, ar fi păcat să nu construiesc una care să îmi rezolve exact această problemă. Nu o aplicație cu zece funcții pe care nu le folosește nimeni, ci una simplă, care să facă bine un lucru: să îmi țină minte antrenamentele și să îmi arate dacă mă îmbunătățesc sau nu. Accesibilă din orice browser, gratuită și, de ce nu, open-source.

1.3 Obiectivele lucrării

Plecând de la problemele identificate mai sus, această lucrare își propune să realizeze următoarele:

1. Proiectarea și implementarea completă a unei aplicații web de tip client-server pentru înregistrarea și gestionarea antrenamentelor de forță;
2. Implementarea unui mecanism de autentificare securizat, cu izolarea strictă a datelor fiecărui utilizator;
3. Realizarea unui modul de antrenament în timp real, cu suport pentru timer de odihnă, calculator de distribuție a plăcilor pe bară și indicatori de intensitate (RIR și 1RM estimat);
4. Construirea unui modul de analiză a progresului bazat pe regresie liniară, cu predicție pe termen scurt și detecție automată a platourilor de performanță;
5. Validarea soluției prin utilizare reală și analiza datelor obținute.

1.4 Contribuții originale

Pe lângă implementarea tehnică propriu-zisă, această lucrare aduce următoarele contribuții originale:

- ▷ Proiectarea unui model de date adaptat specific domeniului antrenamentelor de forță, cu suport pentru seturi de încălzire, RIR per set și asocierea sesiunilor cu șabloane de rutine;
- ▷ Implementarea calculului 1RM estimat prin combinarea a două formule consacrate – Brzycki [3] și Epley [4] – și corelarea rezultatului cu valoarea RIR înregistrată;
- ▷ Un algoritm simplu dar eficient de detecție a platourilor, bazat pe analiza tendinței volumului săptămânal pe o fereastră de trei săptămâni;
- ▷ Un calculator de distribuție a plăcilor pe bară (20 kg), util în practică pentru configurarea rapidă a greutății la un exercițiu.

1.5 Structura lucrării

Lucrarea este împărțită în șase capitole, fiecare construindu-se pe cel anterior.

Capitolul 2 analizează aplicațiile existente pe piață – MyFitnessPal, Strong, Hevy și Jefit – și explică de ce niciuna nu acoperă complet nevoia pe care Gym-Connect o întâmpină.

Capitolul 3 prezintă toate tehnologiile folosite în proiect – de la React și TypeScript până la Prisma, PostgreSQL pe Neon și Clerk – și argumentează de ce a fost aleasă fiecare.

Capitolul 4 descrie arhitectura sistemului: cum sunt organizate datele în baza de date, cum funcționează API-ul REST și cum este gestionată autentificarea.

Capitolul 5 prezintă aplicația așa cum arată în realitate – cu capturi de ecran din fiecare modul și fragmente de cod reprezentative.

Capitolul 6 trage concluziile, discută ce ar putea fi făcut mai bine și propune direcții de dezvoltare pentru viitor.

Capitolul 2

Starea actuală a domeniului

Monitorizarea antrenamentelor fizice reprezintă un domeniu în continuă expansiune, susținut atât de creșterea interesului populației pentru un stil de viață sănătos, cât și de evoluția rapidă a tehnologiei mobile și web. Primele instrumente digitale dedicate acestui scop au apărut odată cu proliferarea smartphone-urilor, începând din jurul anului 2009, și de atunci piața aplicațiilor de fitness a crescut constant. Conform unui raport publicat de Grand View Research [1], piața globală a aplicațiilor de fitness a fost evaluată la peste 1,3 miliarde de dolari în 2022, cu o rată de creștere anuală de aproximativ 17%.

Cu toate acestea, majoritatea soluțiilor existente prezintă limite semnificative: modele de abonament cu funcții esențiale blocate în spatele unui paywall, interfețe supracărcate nepotrivite pentru începători, sau lipsă totală a instrumentelor de analiză a progresului. Acest capitol analizează principalele aplicații existente pe piață, identifică punctele lor forte și limitele, și poziționează soluția propusă – Gym-Connect – în raport cu acestea.

2.1 Aplicații existente de monitorizare a antrenamentelor

2.1.1 MyFitnessPal

MyFitnessPal [5] este una dintre cele mai populare aplicații de fitness din lume, lansată în 2005 și achiziționată de Under Armour în 2015. Aplicația se axează în principal pe **monitorizarea nutriției**, oferind o bază de date cu peste 14 milioane de alimente și un calculator detaliat de calorii și macronutrienți.

Din perspectiva antrenamentelor fizice, MyFitnessPal oferă funcționalități de bază: înregistrarea exercițiilor, estimarea caloriilor arse și integrarea cu dispozitive wearable (Fitbit, Apple Watch, Garmin). Totuși, aplicația nu este proiectată pentru **monitorizarea detaliată a forței** – nu există suport nativ pentru înregistrarea greutăților, seturilor, repetițiilor sau indicatorilor

de intensitate de tip RIR (*Reps in Reserve*) sau 1RM (*one-repetition maximum*).

Puncte forte:

- ▷ bază de date nutritională extrem de bogată;
- ▷ integrare cu numeroase dispozitive și aplicații terțe;
- ▷ comunitate activă și recunoscută la nivel global.

Limitări:

- ▷ funcționalitățile avansate sunt disponibile doar în varianta *Premium*, cu abonament lunar sau anual;
- ▷ nu oferă suport pentru monitorizarea forței și a progressive overload;
- ▷ interfața este supracărcată și poate descuraja utilizatorii începători.

2.1.2 Strong App

Strong [6] este o aplicație dedicată exclusiv antrenamentelor de forță, disponibilă pe iOS și Android. Lansată în 2014, aceasta permite utilizatorilor să creeze rutine personalizate, să înregistreze seturi cu greutate și repetiții și să vizualizeze progresul prin grafice simple. Strong a devenit aplicația de referință pentru mulți sportivi amatori care practică antrenamente de tip *weightlifting*.

Aplicația dispune de un timer de odihnă între seturi, un istoric detaliat al antrenamentelor și un calculator de 1RM bazat pe formula Epley. Interfața este curată și intuitivă, însă versiunea gratuită este sever limitată: utilizatorii pot crea **maximum 3 rutine**, iar funcțiile analitice avansate sunt disponibile exclusiv în varianta Pro, la un preț de aproximativ 10 dolari/lună [6].

Puncte forte:

- ▷ interfață simplă și focusată pe antrenamentele de forță;
- ▷ timer integrat de odihnă și calculator 1RM;
- ▷ istoric detaliat al sesiunilor.

Limitări:

- ▷ limita de 3 rutine în varianta gratuită este o barieră semnificativă;
- ▷ disponibilă doar ca aplicație nativă (iOS/Android), nu și ca aplicație web;
- ▷ analitică avansată blocată în spatele abonamentului Pro;
- ▷ codul sursă este proprietar, fără posibilitate de extindere sau adaptare.

2.1.3 Hevy

Hevy [7] este o aplicație mai recentă, lansată în 2020, care a câștigat rapid popularitate datorită interfeței moderne și funcționalităților sociale. Hevy permite înregistrarea antrenamentelor, crearea de rutine, vizualizarea progresului și **urmărirea activității prietenilor** – o componentă de rețea socială dedicată fitness-ului.

Aplicația oferă grafice de progres per exercițiu, statistici săptămânale și notificări de reminder. Hevy are și un API public documentat, ceea ce permite integrarea cu alte instrumente. Varianta gratuită este mai generoasă decât a competitorilor, însă funcțiile premium (rutine nelimitate, analize avansate) necesită abonament [7].

Puncte forte:

- ▷ interfață modernă și plăcută vizual;
- ▷ componentă socială pentru urmărirea progresului prietenilor;
- ▷ API public documentat.

Limitări:

- ▷ disponibilă predominant ca aplicație nativă;
- ▷ funcții analitice limitate în varianta gratuită;
- ▷ nu oferă instrumente avansate de predicție a performanței sau detecție a platourilor;
- ▷ codul sursă este proprietar.

2.1.4 Jefit

Jefit [8] este una dintre cele mai vechi aplicații de antrenament, disponibilă din 2010. Aceasta oferă o bibliotecă extinsă de exerciții cu instrucțiuni animate, planuri de antrenament predefinite și posibilitatea de a urmări progresul muscular. Jefit se adresează atât începătorilor, cât și sportivilor avansați, oferind conținut educațional integrat.

Totuși, Jefit suferă de aceeași problemă structurală ca majoritatea competitorilor: funcționalitățile esențiale sunt disponibile gratuit, dar planurile personalizate, analizele detaliate și sincronizarea pe mai multe dispozitive necesită un abonament Elite, evaluat la aproximativ 8 dolari/lună [8].

Puncte forte:

- ▷ bibliotecă extinsă de exerciții cu instrucțiuni animate;
- ▷ planuri predefinite potrivite pentru începători;
- ▷ disponibilă atât pe mobil, cât și ca aplicație web.

Limitări:

- ▷ interfață învechită și neintuitivă;
- ▷ funcții analitice avansate în spatele abonamentului;
- ▷ codul sursă este proprietar.

2.2 Analiză comparativă

Tabelul 2.1 prezintă o comparație sintetică între aplicațiile analizate și soluția propusă – Gym-Connect.

Din analiza tabelului se observă că niciuna dintre aplicațiile existente nu oferă în varianta gratuită toate funcționalitățile necesare unui sportiv care dorește să aplice principiile

Tabelul 2.1: Comparație între aplicațiile de monitorizare a antrenamentelor

Caracteristică	MyFitnessPal	Strong	Hevy	Jefit	Gym-Connect
Gratuit complet	Nu	Partial	Partial	Partial	Da
Aplicație web	Da	Nu	Partial	Da	Da
Tracker forță	Nu	Da	Da	Da	Da
Calcul 1RM	Nu	Da	Nu	Nu	Da
Deteție platouri	Nu	Nu	Nu	Nu	Da
Predicție progres	Nu	Nu	Nu	Nu	Da
Calculator plăci	Nu	Nu	Nu	Nu	Da
RIR per set	Nu	Nu	Nu	Nu	Da
Open-source	Nu	Nu	Nu	Nu	Da

progressive overload în mod riguros. Mai mult, instrumente avansate precum detecția platourilor de performanță sau predicția progresului pe bază de regresie liniară lipsesc complet din oferta competitorilor analizați.

2.3 Poziționarea Gym-Connect

Gym-Connect se poziționează ca o alternativă **open-source**, **gratuită** și **accesibilă din browser**, specific proiectată pentru sportivi care practică antrenamente de forță și doresc să își gestioneze progresul într-un mod structurat.

Principalele elemente de diferențiere față de soluțiile existente sunt:

1. **Accesibilitate totală** – aplicația funcționează direct din browser, fără instalare, pe orice dispozitiv cu conexiune la internet. Acest lucru elimină bariera de acces specifică aplicațiilor native.
2. **Fără restricții de tip paywall** – toate funcționalitățile sunt disponibile gratuit. Utilizatorul nu este limitat la un număr fix de rutine și nu trebuie să plătească pentru a accesa analizele de progres.
3. **Analitică avansată** – Gym-Connect include un modul de analiză a progresului bazat pe regresie liniară, cu predicția evoluției 1RM pe un orizont de 4 săptămâni și detecție automată a platourilor de performanță. Aceste funcționalități lipsesc complet din oferta competitorilor analizați.
4. **Indicatori de intensitate integrați** – calculul 1RM estimat prin combinarea formulelor Brzycki și Epley [3], măsurarea intensității prin RIR (*Reps in Reserve*) și calculatorul de plăci reprezintă instrumente profesionale rare în aplicațiile gratuite.
5. **Cod sursă deschis** – spre deosebire de toți competitorii analizați, Gym-Connect este un proiect open-source, disponibil public pe GitHub [9], permișând inspecția codului, contribuții și adaptare la nevoi specifice.

Gym-Connect nu își propune să concureze cu aplicații de talia MyFitnessPal pe segmentul nutriției sau cu Hevy pe cel al rețelelor sociale. Scopul său este precis delimitat: să ofere sportivului amator un instrument complet, simplu și gratuit pentru **gestionarea antrenamentelor de forță și analiza progresului pe termen lung**.

Capitolul 3

Tehnologii și elemente teoretice utilizate

Alegerea stivei tehnologice a fost una dintre primele și cele mai importante decizii ale acestui proiect. Criteriul principal a fost stabilitatea: nu am căutat să folosesc ceva nou doar pentru că e la modă, ci tehnologii mature, bine documentate și cu o comunitate activă. Un alt criteriu a fost compatibilitatea reciprocă – toate piesele stivei trebuiau să se integreze natural, fără prea multă configurație manuală. Rezultatul este o stivă omogenă TypeScript atât pe frontend, cât și pe backend, ceea ce a simplificat considerabil dezvoltarea și mentenanța codului.

3.1 React

Atunci când am început să planific front-endul, React [10] era alegerea evidentă. Nu neapărat pentru că e cel mai popular – ci pentru că modelul său de **componente** se potrivește perfect cu modul în care gândesc o interfață: unități independente, fiecare cu responsabilitatea ei, reutilizabile oriunde ai nevoie de ele.

React a fost creat de Meta și lansat public în 2013. La baza lui stă un mecanism numit **Virtual DOM**: în loc să modifice direct pagina HTML la orice schimbare de date, React calculează mai întâi diferența față de starea anterioară și aplică în DOM-ul real doar modificările strict necesare. E un detaliu de implementare, dar are un efect practic important: interfețele rămân fluide chiar și când datele se schimbă frecvent.

O schimbare majoră a venit în 2019, odată cu introducerea **Hook-urilor**. Înainte, orice componentă care avea nevoie de stare trebuia scrisă ca o clasă – verbos și greu de urmărit. Hook-urile (`useState`, `useEffect` și celelalte) au rezolvat asta elegant, allowând gestionarea stării direct în funcții simple. În Gym-Connect, toate componentele sunt funcționale și folosesc Hook-uri – nu există nicio clasă în cod.

3.2 TypeScript

Dacă React a fost o alegere de tip „*așa se face*”, TypeScript [11] a fost mai degrabă o alegere din experiență personală. Am scris JavaScript pur destul de mult și știu cum arată un bug găsit în producție dintr-o comparare de tipuri greșită. TypeScript elimină întreaga categorie de astfel de probleme.

TypeScript este un superset al JavaScript-ului, dezvoltat de Microsoft și lansat în 2012. Practic, orice cod JavaScript valid este și TypeScript valid – diferența este că TypeScript adaugă un strat de **tipizare statică**, opțională ca granularitate, dar extrem de utilă atunci când proiectul crește. Compilatorul detectează erorile înainte de rulare, nu după.

În contextul Gym-Connect, avantajul concret a fost următorul: schema Prisma generează automat tipuri TypeScript pentru modelele de date. Asta înseamnă că atunci când frontenidul consumă răspunsul API-ului, editorul știe exact ce proprietăți există pe obiectul primit, semnalezând imediat orice acces incorect.

3.3 Vite

Vite [12] este instrumentul care compilează și servește aplicația în timpul dezvoltării. L-am ales în locul Create React App (care a și fost întrerupt între timp) dintr-un motiv simplu: este remarcabil de rapid.

Creat de Evan You (autorul Vue.js) și lansat în 2020, Vite funcționează diferit față de bundler-ele clasice. În modul de dezvoltare, nu procesează întreaga aplicație la pornire – servește fișierele direct folosind modulele ES native ale browserului și transformă fiecare fișier **la cerere**, abia când browserul îl solicită. Efectul practic: serverul de dezvoltare pornește în sub o secundă, indiferent de dimensiunea proiectului.

Un alt lucru apreciat în practică este **HMR-ul** (*Hot Module Replacement*): modificările de cod se reflectă în browser în milisecunde, fără să se piardă starea aplicației.

3.4 Express.js

Pentru server, am ales Express.js [13] – nu pentru că e singurul framework Node.js, ci pentru că e cel mai simplu. Express își cunoaște locul: oferă rutare HTTP, un sistem de middleware și gestionarea cererilor, atât. Restul îl alegi tu.

Lansat în 2010, Express a devenit practic sinonimul cu „server Node.js”. Filosofia lui de tip „*minimalist*” înseamnă că nu primești o structură impusă – îți organizezi singur codul cum consideri că se potrivește cel mai bine. În Gym-Connect asta a însemnat un fișier `app.ts` care înregistrează două grupuri de rute (`/api/splits` și `/api/sessions`), un middleware de autentificare și un handler global de erori. Simplu și ușor de urmărit.

3.5 Prisma ORM

Interacțiunea cu baza de date este gestionată prin Prisma [14], un ORM (*Object-Relational Mapping*) de generație nouă pentru TypeScript. Am încercat în trecut Sequelizee și am rămas cu impresia unui instrument greoi, cu o documentație imensă și erori greu de depanat. Prisma este opusul: schema se definește într-un fișier simplu și clar (`schema.prisma`), clientul generat automat este complet tipizat, iar interogările se scriu într-un stil aproape lizibil ca proză.

Cel mai important avantaj practic: dacă scrii o interogare care accesează un câmp care nu există în schema bazei de date, compilatorul TypeScript semnalizează eroarea înainte să rulezi codul. Am prins astfel mai multe greșeli de tastare înainte să ajungă în execuție.

Prisma vine și cu **Prisma Studio**, o interfață web locală prin care poți vedea și edita direct datele din baza de date – extrem de util în faza de testare.

3.6 PostgreSQL și Neon DB

Baza de date folosită este **PostgreSQL** [15], găzduită pe platforma cloud **Neon DB** [16]. PostgreSQL este o alegere clasică pentru aplicații cu date structurate: există de peste 35 de ani, e open-source, respecta standardele SQL și are o reputație solidă de stabilitate.

Neon DB a fost ales pentru că rezolvă elegant problema costurilor în faza de dezvoltare. Fiind o platformă *serverless*, baza de date se oprește automat în perioadele de inactivitate (*auto-suspend*) și repornește la prima cerere. Planul gratuit este generos pentru un proiect de tip portofoliu sau licență: 0.5 GB stocare și 100 CU-ore de calcul pe lună. Am ales regiunea Frankfurt (EU Central) pentru latență redusă în zona Europeană.

3.7 Clerk

Autentificarea este unul dintre lucrurile pe care prefer să nu le implementez de la zero dacă există o soluție matură. Implementarea corectă a autentificării – hashing parole, gestionare sesiuni, refresh tokenuri, protecție brute-force – este complexă și ușor de greșit. Clerk [17] rezolvă toate acestea într-un pachet.

Clerk este o platformă de tip *Authentication as a Service*, lansată în 2021. Oferă componente UI gata de folosit pentru login și register, gestionează sesiunile, suportă autentificarea socială și vine cu SDK-uri separate pentru frontend (`@clerk/react`) și backend (`@clerk/express`). Integrarea cu Express se reduce practic la adăugarea unui middleware și a unui apel de funcție pentru extragerea `userId`-ului din cerere.

Un aspect important: Clerk atribuie fiecărui utilizator autentificat un ID unic (`userId`). În Gym-Connect, toate datele din baza de date sunt legate de acest ID, ceea ce garantează că

un utilizator nu poate vedea sau modifica datele altcuiva.

3.8 Tailwind CSS

Pentru stilizarea interfeței am ales Tailwind CSS [18], un framework de tip *utility-first* care funcționează diferit față de Bootstrap sau alte framework-uri clasice. În loc să ofere componente gata construite (un buton, un card, un modal), Tailwind oferă clase utilitare de nivel scăzut pe care le combini direct în HTML/JSEX ca să obții exact ce îți trebuie.

La început părea ciudat să scrii `flex items-center gap-4 rounded-lg bg-gray-900` direct pe un element, dar după câteva zile de practică înțelegi avantajul: nu mai sari între fișiere CSS și componente, totul este într-un singur loc. Iar datorită compilatorului JIT, CSS-ul final conține exclusiv clasele folosite efectiv, rezultând bundle-uri mici.

3.9 Recharts

Modulul de analitică are nevoie de grafice, și pentru asta am ales Recharts [19] – o bibliotecă de vizualizare construită peste D3.js, dar expusă ca componente React native. Am preferat-o în fața D3.js pur pentru că D3 manipulează direct DOM-ul, ceea ce nu se integrează bine cu modelul React. Recharts funcționează declarativ, ca orice altă componentă JSX – defineai graficul ca structură, și Recharts se ocupă de tot restul.

În practică am folosit două tipuri de grafice: un grafic de tip *arie* pentru volumul săptămânal și un grafic de tip *linie* cu trei serii pentru evoluția 1RM estimat (valori reale, trend și predicție). Ambele sunt responsive și au tooltip-uri interactive.

3.10 Zod

Zod [20] este biblioteca pe care o folosesc pentru validarea datelor primite de server. Fără validare, un utilizator rău intenționat (sau pur și simplu un bug pe frontend) ar putea trimite date malformate care să ajungă în baza de date.

Zod permite definirea unei scheme – practic o descriere a cum ar trebui să arate un obiect valid – și parsarea datelor primite față de acea schemă. Dacă datele nu corespund, Zod aruncă o eroare cu un mesaj clar despre ce câmp e greșit. Un avantaj important în contextul TypeScript: din orice schemă Zod poți extrage automat tipul corespunzător cu `z.infer<typeof schema>`, fără să duplici definiția.

3.11 React Router

Navigația între paginile aplicației este gestionată de React Router [21], biblioteca standard pentru rutare în aplicații React. Cum Gym-Connect este o aplicație de tip *SPA* (*Single Page Application*), tranziția între pagini se face fără reîncărcarea paginii – URL-ul se schimbă, componenta afișată se schimbă, dar serverul nu e contactat pentru HTML nou.

Am folosit React Router și pentru a construi componenta `ProtectedRoute`: o rută care, înainte să afișeze conținutul, verifică dacă utilizatorul este autentificat prin Clerk. Dacă nu, îl redirecționează către pagina de login.

3.12 React Hook Form

În aplicație există mai multe formulare: creare și editare rutină, adaugare exerciții. Pentru gestionarea lor am ales React Hook Form [22], din motive de performanță.

Într-un formular gestionat clasic cu `useState`, orice tastare declanșează o re-randare a componentei. React Hook Form evită asta prin abordarea *uncontrolled*: înregistrează referințe la elementele HTML native și citește valorile doar la submit sau la validare. Rezultatul e un formular mai fluid, mai ales pe dispozitive mai lente. Integrarea nativă cu Zod prin `@hookform/resolvers` înseamnă că schema de validare scrisă o singură dată este re-utilizată automat și pentru validarea formularului.

Capitolul 4

Soluția propusă și arhitectura sistemului

Acest capitol descrie arhitectura generală a aplicației Gym-Connect, modelul de date, API-ul REST implementat și mecanismele de securitate utilizate. Deciziile de proiectare sunt argumentate în raport cu cerințele funcționale și nefuncționale ale sistemului.

4.1 Arhitectura generală

Gym-Connect adoptă o arhitectură clasică de tip **client-server**, în care cele două componente principale – interfața utilizator (frontend) și serverul API (backend) – rulează independent și comunică prin cereri HTTP.

Sistemul este compus din trei niveluri distincte:

1. **Nivelul de prezentare (Frontend)** – aplicația React care rulează în browserul utilizatorului. Aceasta gestionează interfața utilizator, starea locală a aplicației și comunică cu serverul prin cereri HTTP autentificate.
2. **Nivelul de logică de business (Backend)** – serverul Express.js care rulează pe Node.js. Acesta primește cererile de la client, le validează, aplică logica de business și interacționează cu baza de date prin Prisma ORM.
3. **Nivelul de date (Baza de date)** – instanța PostgreSQL găzduită pe platforma Neon DB. Stochează toate datele aplicației: rutine de antrenament, sesiuni și seturi individuale.

Fluxul standard al unei cereri este următorul: browserul utilizatorului trimite o cerere HTTP către serverul Express, însoțită de un token de autentificare Clerk. Serverul validează tokenul, procesează cererea, interoghează baza de date prin Prisma și returnează răspunsul în format JSON către client.

În modul de dezvoltare, Vite oferă un **proxy** către serverul backend, astfel încât cererile de pe `localhost:5173/api/*` sunt redirijate automat către `localhost:3001/api/*`, evitând problemele de CORS în timpul dezvoltării.

4.2 Modelul de date

Schema bazei de date este definită declarativ în fișierul `prisma/schema.prisma` și este compusă din trei modele principale, interconectate prin relații de tip *one-to-many*.

4.2.1 Modelul SplitTemplate

Modelul `SplitTemplate` reprezintă un **șablon de rutină de antrenament** – un plan pre-definit care descrie ce exerciții, câte seturi și ce interval de repetiții trebuie efectuate. Un utilizator poate crea oricâte rutine, iar fiecare rutină aparține exclusiv utilizatorului care a creat-o.

Tabelul 4.1: Câmpurile modelului `SplitTemplate`

Câmp	Tip	Descriere
<code>id</code>	String (UUID)	Identificator unic generat automat
<code>userId</code>	String	ID-ul utilizatorului proprietar (Clerk)
<code>name</code>	String	Numele rutinei
<code>description</code>	String	Descriere opțională
<code>exercises</code>	JSON	Lista exercițiilor cu seturi și repetiții
<code>createdAt</code>	DateTime	Data creării
<code>updatedAt</code>	DateTime	Data ultimei modificări

Câmpul `exercises` stochează o structură JSON care conține un array de obiecte, fiecare descriind un exercițiu din rutină cu următoarele proprietăți: `exerciseId` (identificatorul exercițiului din baza de date locală), `targetSets` (numărul de seturi țintă), `targetRepRange` (intervalul de repetiții, ex. 8–12) și `order` (ordinea exercițiului în rutină). Alegerea stocării în format JSON a fost motivată de faptul că structura exercițiilor dintr-o rutină este variată și nu necesită interogări relaționale independente.

4.2.2 Modelul WorkoutSession

Modelul `WorkoutSession` reprezintă o **sesiune de antrenament efectuată** – o înregistrare a unui antrenament real, cu data, durata și seturile efectuate. Fiecare sesiune poate fi opțional asociată cu o rutină de antrenament (`SplitTemplate`), dar poate exista și independent (antrenament liber). Relația cu `SplitTemplate` este de tip *many-to-one*, cu comporta-

ment `SetNull` la ștergere – dacă o rutină este ștearsă, sesiunile asociate nu sunt șterse, ci rămân cu `splitId = null`.

Tabelul 4.2: Câmpurile modelului `WorkoutSession`

Câmp	Tip	Descriere
<code>id</code>	String (CUID)	Identificator unic
<code>userId</code>	String	ID-ul utilizatorului (Clerk)
<code>splitId</code>	String?	Referință opțională la rutină
<code>splitName</code>	String	Numele rutinei la momentul sesiunii
<code>date</code>	DateTime	Data și ora antrenamentului
<code>durationSeconds</code>	Int	Durata în secunde
<code>notes</code>	String?	Note opționale
<code>createdAt</code>	DateTime	Data înregistrării

Câmpul `splitName` stochează numele rutinei la momentul efectuării sesiunii. Această decizie de proiectare asigură că istoricul rămâne corect chiar dacă utilizatorul redenumeste sau șterge rutina ulterior.

4.2.3 Modelul `WorkoutSet`

Modelul `WorkoutSet` reprezintă **un set individual** dintr-o sesiune de antrenament – cea mai granulară unitate de date a aplicației. Fiecare sesiune conține unul sau mai multe seturi, fiecare corespunzând unui exercițiu specific. Relația cu `WorkoutSession` este de tip *many-to-one* cu comportament *Cascade* – ștergerea unei sesiuni atrage și ștergerea tuturor seturilor asociate.

Tabelul 4.3: Câmpurile modelului `WorkoutSet`

Câmp	Tip	Descriere
<code>id</code>	String (CUID)	Identificator unic
<code>sessionId</code>	String	Referință la sesiunea părinte
<code>exerciseId</code>	String	ID-ul exercițiului
<code>setOrder</code>	Int	Ordinea setului în sesiune
<code>weight</code>	Float	Greutatea utilizată (kg)
<code>reps</code>	Int	Numărul de repetiții efectuate
<code>rir</code>	Int	Reps in Reserve (0–10)
<code>isWarmup</code>	Boolean	Marcaj set de încălzire

Câmpul `rir` (*Reps in Reserve*) reprezintă numărul estimat de repetiții rămase până la cedare. Un RIR de 0 indică epuizare musculară completă, în timp ce un RIR de 3 indică că sportivul putea efectua încă 3 repetiții. Acest indicator este utilizat în modulul de analitică pentru calculul 1RM estimat și aprecierea intensității antrenamentului.

4.2.4 Relațiile dintre modele

Cele trei modele formează o ierarhie clară:

- ▷ Un utilizator (identificat prin `userId` Clerk) poate deține orice număr de `SplitTemplate`-uri și `WorkoutSession`-uri.
- ▷ O `WorkoutSession` poate fi asociată cu cel mult un `SplitTemplate` (opțional).
- ▷ O `WorkoutSession` conține unul sau mai multe `WorkoutSet`-uri.

Izolarea datelor între utilizatori este garantată la nivelul aplicației: toate interogările Prisma includ filtrul `where: { userId }` extras din tokenul de autentificare Clerk, astfel încât un utilizator nu poate accesa datele altui utilizator.

4.3 API-ul REST

Serverul Express expune un API REST cu două grupuri de endpoint-uri, ambele protejate prin middleware-ul de autentificare Clerk.

4.3.1 Endpoint-uri pentru rutine (/api/splits)

Tabelul 4.4: Endpoint-urile API pentru gestionarea rutinelor

Metodă	Rută	Descriere
GET	/api/splits	Returnează toate rutinele utilizatorului
POST	/api/splits	Creează o rutină nouă
PUT	/api/splits/:id	Actualizează o rutină existentă
DELETE	/api/splits/:id	Șterge o rutină

Endpoint-ul `POST /api/splits` validează corpul cererii cu schema Zod, verificând că rutina conține un nume nenul, o descriere opțională și cel puțin un exercițiu cu `exerciseId`, `targetSets` și `targetRepRange` valide. Endpoint-urile `PUT` și `DELETE` verifică suplimentar că resursa solicitată aparține utilizatorului curent, returnând HTTP 404 în caz contrar.

4.3.2 Endpoint-uri pentru sesiuni (/api/sessions)

Tabelul 4.5: Endpoint-urile API pentru gestionarea sesiunilor

Metodă	Rută	Descriere
GET	/api/sessions	Returnează toate sesiunile utilizatorului
POST	/api/sessions	Salvează o sesiune nouă cu toate seturile

Endpoint-ul `POST /api/sessions` primește într-o singură cerere toate datele sesiunii: metadatele (dată, durată, note, rutină asociată) și lista completă a seturilor efectuate. Crearea sesiunii și a seturilor asociate se face într-o singură tranzacție Prisma (`create` cu `sets: { create: [...] }`), garantând atomicitatea operației – fie se salvează totul, fie nimic.

Endpoint-ul `GET /api/sessions` returnează sesiunile ordonate descendent după dată, împreună cu seturile asociate ordonate după `setOrder`.

4.4 Autentificarea și securitatea

Autentificarea în Gym-Connect este gestionată integral de serviciul **Clerk**, care furnizează atât componentele UI de login/register, cât și mecanismul de validare a sesiunilor pe server.

4.4.1 Fluxul de autentificare

Fluxul de autentificare funcționează în următorii pași:

1. Utilizatorul se autentifică prin interfata Clerk (email + parolă sau OAuth).
2. Clerk emite un **JWT** (*JSON Web Token*) semnat cu cheia privată a aplicației.
3. La fiecare cerere HTTP către server, clientul React include automat acest token în header-ul `Authorization`.
4. Pe server, middleware-ul `clerkMiddleware()` validează tokenul și populează obiectul cererii cu datele utilizatorului autentificat.
5. Middleware-ul `requireAuth` extrasă `userId` din cerere; dacă acesta lipsește, returnează HTTP 401.

4.4.2 Middleware-ul de autentificare

Middleware-ul `requireAuth` este aplicat global pe toate rutele API și are următoarea logică:

1. Extrage `userId` din contextul Clerk al cererii curente folosind funcția `getAuth(req)`.
2. Dacă `userId` este absent (utilizator neautentificat sau token expirat), returnează imediat răspunsul `401 Unauthorized` cu mesajul „*Autentificare necesară*”.
3. Dacă autentificarea este validă, adaugă `userId` pe obiectul cererii și transferă controlul către handler-ul rutei.

Prin această abordare, nicio rută API nu poate fi accesată fără un token valid, iar `userId` este extras exclusiv din tokenul verificat de Clerk, nu din parametrii cererii – eliminând posibilitatea falsificării identității.

4.4.3 Protecția rutelor în frontend

În frontend, rutele private sunt protejate prin componenta `ProtectedRoute`, care utilizează hook-ul `useAuth()` al Clerk pentru a verifica starea de autentificare. Dacă utilizatorul nu este autentificat, este redirijat automat către pagina de autentificare. În caz contrar, componenta afișează conținutul paginii solicitate.

4.5 Diagrama Use-Case

Diagrama Use-Case din figura ?? ilustrează principalele interacțiuni ale utilizatorului cu aplicația Gym-Connect.

Utilizatorul neautentificat poate accesa exclusiv pagina de start și paginile de autentificare. După autentificare, acesta poate efectua următoarele acțiuni principale:

- ▷ **Gestiona rutine** – creare, editare și ștergere rutine de antrenament cu exerciții, seturi și repetiții țintă.
- ▷ **Efectua antrenament** – pornire sesiune de antrenament bazată pe o rutină existentă, înregistrare seturi cu greutate, repetiții și RIR, utilizare calculator de plăci.
- ▷ **Vizualiza istoricul** – consultarea tuturor sesiunilor anterioare cu detalii complete.
- ▷ **Analiza progresul** – vizualizarea volumului săptămânal, a evoluției 1RM estimat și a avertismentelor de platou.

Capitolul 5

Prezentarea aplicației

5.1 Pagina de start (Landing Page)

5.2 Autentificare și înregistrare

5.3 Gestionarea rutinelor (Splits)

5.3.1 Crearea unei rutine

5.3.2 Selectorul de exerciții

5.4 Sesiunea de antrenament (Workout)

5.4.1 Cronometrul și indicatorul de progres

5.4.2 Gestionarea seturilor

5.4.3 Calculatorul de plăci

5.4.4 Indicatori de intensitate: RIR și 1RM

5.5 Istoricul antrenamentelor (History)

5.6 Modulul de analitică (Analytics)

5.6.1 Graficul volumului săptămânal

5.6.2 Predicția 1RM și regresia liniară

5.6.3 Detecția platourilor

Capitolul 6

Concluzii

6.1 Rezultate obținute

6.2 Limitările soluției actuale

6.3 Direcții de dezvoltare ulterioară

Referințe bibliografice

- [1] Grand View Research. Fitness app market size, share & trends analysis report, 2023–2030. <https://www.grandviewresearch.com/industry-analysis/fitness-app-market>, 2023. Accesat: martie 2026.
- [2] Vladimir M. Zatsiorsky and William J. Kraemer. *Science and Practice of Strength Training*. Human Kinetics, Champaign, IL, 2nd edition, 2006.
- [3] Matt Brzycki. Strength testing — predicting a one-rep max from reps-to-fatigue. *Journal of Physical Education, Recreation & Dance*, 64(1):88–90, 1993.
- [4] Boyd Epley. *Poundage Chart*. University of Nebraska, Lincoln, NE, 1985.
- [5] MyFitnessPal, Inc. Myfitnesspal — calorie counter and diet tracker. <https://www.myfitnesspal.com>, 2024. Accesat: martie 2026.
- [6] Anvil Labs Ltd. Strong — workout tracker gym log. <https://www.strong.app>, 2024. Accesat: martie 2026.
- [7] Hevy Inc. Hevy — workout tracker & planner. <https://www.hevyapp.com>, 2024. Accesat: martie 2026.
- [8] JEFIT Inc. Jefit — workout planner gym log. <https://www.jefit.com>, 2024. Accesat: martie 2026.
- [9] Andrei-Robert Dode. Gym-connect — aplicație web pentru gestionarea antrenamentelor fizice. <https://github.com/SonDode/Gym-connect>, 2026. Accesat: martie 2026.
- [10] Meta Platforms, Inc. React — the library for web and native user interfaces. <https://react.dev>, 2024. Accesat: martie 2026.
- [11] Microsoft Corporation. Typescript — javascript with syntax for types. <https://www.typescriptlang.org>, 2024. Accesat: martie 2026.
- [12] Evan You and Vite Contributors. Vite — next generation frontend tooling. <https://vitejs.dev>, 2024. Accesat: martie 2026.

-
- [13] OpenJS Foundation. Express.js — fast, unopinionated, minimalist web framework for node.js. <https://expressjs.com>, 2024. Accesat: martie 2026.
 - [14] Prisma Data, Inc. Prisma — next-generation orm for node.js & typescript. <https://www.prisma.io/docs>, 2024. Accesat: martie 2026.
 - [15] The PostgreSQL Global Development Group. Postgresql 17 documentation. <https://www.postgresql.org/docs/17/>, 2024. Accesat: martie 2026.
 - [16] Neon Inc. Neon — serverless postgres. <https://neon.tech/docs>, 2024. Accesat: martie 2026.
 - [17] Clerk, Inc. Clerk — authentication and user management. <https://clerk.com/docs>, 2024. Accesat: martie 2026.
 - [18] Adam Wathan and Tailwind Labs Inc. Tailwind css — rapidly build modern websites. <https://tailwindcss.com/docs>, 2024. Accesat: martie 2026.
 - [19] Recharts Group. Recharts — redefined chart library built with react and d3. <https://recharts.org/en-US>, 2024. Accesat: martie 2026.
 - [20] Colinhacks. Zod — typescript-first schema validation with static type inference. <https://zod.dev>, 2024. Accesat: martie 2026.
 - [21] Remix Software, Inc. React router v6 documentation. <https://reactrouter.com/en/main>, 2024. Accesat: martie 2026.
 - [22] Bill Liu. React hook form — performant, flexible and extensible forms. <https://react-hook-form.com>, 2024. Accesat: martie 2026.